

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ДОНСКОЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(ДГТУ)**

КОНСПЕКТ ЛЕКЦИИ № 1

**АЛГОРИТМ, СПЕЦИФИКАЦИЯ, МОДЕЛЬ
ВЫЧИСЛЕНИЙ.
КОРРЕКТНОСТЬ И ИНВАРИАНТЫ**

по дисциплине «Алгоритмы и структуры данных»

Направление подготовки: 03.03.01 Прикладные математика и физика
Направленность (профиль): 03.03.01 Прикладные физика и математика
Семестр: 7 · Объем лекции: 2 академических часа
Компетенция: ПК-4 · Индикатор: ПК-4.4

1. Паспорт лекции

Позиция	Содержание
Место в РПД	Раздел 1. Основы алгоритмизации и анализа сложности; первая из двух лекций раздела.
Цель	Сформировать строгое представление об алгоритме как о вычислительном объекте, научить задавать контракт решения и строить доказательство корректности с помощью инварианта.
Планируемый результат	Обучающийся формулирует предусловие и постусловие, выбирает модель вычислений, различает частичную и полную корректность, строит и проверяет инвариант простого цикла.
Связь с ПК-4.4	Использует программные средства осмысленно: сначала фиксирует спецификацию и критерии корректности, затем реализует и тестирует алгоритм.
Предварительная подготовка	Базовые конструкции языка программирования: переменная, присваивание, условие, цикл, функция; математическая логика и метод индукции.
Форма текущего контроля	Мини-задачи по спецификации и инвариантам, ответы на контрольные вопросы, домашнее задание.

2. Планируемые результаты по уровням

Уровень	После лекции обучающийся демонстрирует
Базовый	Распознает вход, выход и ограничения задачи; отличает алгоритм от конкретной программы; воспроизводит определения предусловия, постусловия и инварианта.
Средний	Самостоятельно записывает контракт простого алгоритма; проверяет инициализацию, сохранение и завершение инварианта; находит граничные случаи.
Продвинутый	Проектирует спецификацию для обработки физико-математических данных; обосновывает выбранную модель вычислений; строит контрпример к некорректному утверждению и формулирует ограничения доказательства.

3. Рекомендуемая организация 90 минут

Этап	Время	Деятельность
Проблемная постановка	0–8 мин	Почему работающая на примерах программа еще не является доказанно корректным решением.
Алгоритм и задача	8–22 мин	Понятия задачи, экземпляра, входа, выхода, допустимости результата.
Спецификация	22–38 мин	Предусловия, постусловия, контракт; разбор примера с порогом измерения.
Модель вычислений	38–50 мин	РАМ-модель, базовые операции и границы применимости.
Корректность	50–62 мин	Частичная и полная корректность, тройка Хоара.
Инвариант цикла	62–82 мин	Построение и доказательство инварианта на сквозном примере.
Закрепление	82–90 мин	Мини-задача, рефлексия, домашнее задание.

4. Зачем алгоритму спецификация

Вычислительная задача обычно формулируется как отношение между допустимыми входами и допустимыми результатами. Программа — лишь одна реализация этого отношения на конкретном языке и в конкретной среде. Без явной спецификации невозможно строго ответить, что считать правильным результатом, какие входы допустимы и какие случаи реализация обязана обработать.

Ключевая мысль. тесты показывают наличие ошибок на выбранных примерах, а доказательство связывает поведение алгоритма со спецификацией для всего заявленного класса входов.

В физико-математических приложениях неоднозначность особенно опасна. Например, фраза «найти превышение порога» не определяет, требуется ли первое превышение, максимальное превышение или число превышений; как обрабатывать *NaN*; считается ли равенство порогу превышением; в каких единицах заданы данные.

5. Алгоритм, программа, задача и экземпляр

5.1. Алгоритм

Алгоритм — конечное, однозначно интерпретируемое описание последовательности вычислительных действий, которое для каждого допустимого входа должно привести к результату, удовлетворяющему спецификации. Такое определение объединяет четыре требования:

- определенность: на каждом шаге ясно, какое действие выполняется далее;
- массовость: описание относится к классу экземпляров, а не к одному набору чисел;
- результативность: при выполнении условий применимости получается результат;
- эффективная выполнимость: каждый элементарный шаг реализуем в выбранной модели вычислений.

5.2. Задача и экземпляр

Задача задает общий класс требований. Экземпляр задачи фиксирует конкретные входные данные. Например, задача поиска первого превышения задается для произвольной последовательности измерений, а массив $[0,18; 0,21; 0,67; 0,52]$ при пороге $0,50$ является отдельным экземпляром.

Понятие	Вопрос	Пример
Задача	Что требуется сделать для любого допустимого входа?	Найти индекс первого измерения, не меньшего порога.
Экземпляр	С какими конкретными данными работаем?	$x = [0,18; 0,21; 0,67; 0,52]$, $\tau = 0,50$.
Алгоритм	Какая конечная процедура строит результат?	Последовательно просматривать элементы до первого $x[i] \geq \tau$.
Программа	Как алгоритм реализован в выбранной среде?	Функция Python/C++ с обработкой типа данных и ошибок ввода.

6. Спецификация как контракт

Спецификация отделяет требуемое поведение от способа его получения. Для функции ее удобно записывать через предусловие P , описывающее допустимый вход, и постусловие Q , описывающее обязательные свойства результата.

Контракт. если перед запуском выполнено предусловие P и алгоритм завершился, после завершения должно быть истинно постусловие Q . Для полной корректности дополнительно требуется доказать завершение.

6.1. Требования к хорошей спецификации

- полнота: перечислены все существенные входы, выходы и ограничения;
- непротиворечивость: для допустимого входа существует допустимый результат;
- проверяемость: свойства результата можно логически или экспериментально проверить;
- независимость от реализации: спецификация не навязывает структуру данных без необходимости;
- явное описание граничных случаев: пустой вход, повторения, предельные значения, специальные числовые значения.

6.2. Сквозной пример: первое достижение порога

Вход: массив действительных измерений $x[0..n-1]$ и порог τ . Результат g : индекс первого элемента, удовлетворяющего $x[g] \geq \tau$, либо -1 , если такого элемента нет.

Предусловие P : $n \geq 0$; доступны элементы $x[0], \dots, x[n-1]$; значения принадлежат согласованной числовой шкале; политика обработки NaN задана отдельно.

Постусловие Q :

если $g = -1$, то для любого j из диапазона $0 \leq j < n$ выполняется $x[j] < \tau$;

если $g \geq 0$, то $0 \leq g < n$, $x[g] \geq \tau$ и для любого $j < g$ выполняется $x[j] < \tau$.

Вторая часть постусловия содержит не только факт превышения, но и минимальность индекса. Возврат любого подходящего индекса нарушил бы слово «первого».

7. Модель вычислений

Для анализа необходимо договориться, какие действия считаются элементарными и какова их стоимость. В базовом курсе используется RAM-модель: память представляется набором адресуемых ячеек, а чтение, запись, арифметическая операция над машинным словом и сравнение имеют постоянную стоимость.

Допущение RAM-модели	Что оно дает	Когда требуется уточнение
Произвольный доступ к памяти за $O(1)$	Позволяет сравнивать массив и последовательный просмотр.	Иерархия кэшей, внешняя память, распределенное хранение.
Операция над словом за $O(1)$	Упрощает подсчет сравнений и присваиваний.	Длинная арифметика, символьные вычисления, произвольная точность.
Последовательное выполнение	Фиксирует однозначный порядок шагов.	Параллельные алгоритмы, GPU, асинхронные вычисления.
Единичная стоимость ввода не учитывается	Отделяет алгоритмическое ядро от интерфейса.	Потоковые данные, сеть, дисковый ввод-вывод.

Модель не обязана полностью повторять аппаратную платформу. Она должна быть достаточно точной для сравнения вариантов и явно оговаривать упрощения. В следующей лекции эта модель станет основой асимптотических оценок.

8. Корректность алгоритма

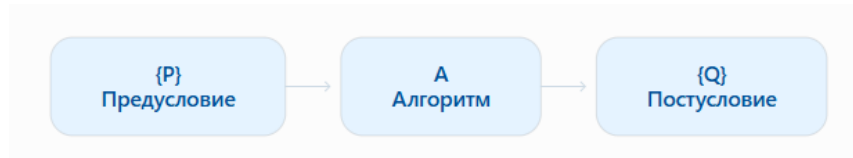
8.1. Частичная и полная корректность

Вид утверждения	Содержание
Частичная корректность	Если алгоритм запущен на входе, удовлетворяющем P , и завершился, то его результат удовлетворяет Q .
Завершаемость	Для каждого входа из области P алгоритм выполняет конечное число шагов.
Полная корректность	Одновременно доказаны частичная корректность и завершаемость.

8.2. Тройка Хоара (логический контракт)

Запись $\{P\} A \{Q\}$ читается так: если перед выполнением алгоритма или фрагмента A истинно P и A завершился, после выполнения истинно Q . Для последовательности операторов доказательство строится от локальных условий к общему контракту.

Схема логического контракта



- P – условия, истинные до выполнения алгоритма.
- A – алгоритм или программный фрагмент.
- Q – свойства результата после завершения A .

Смысл: если перед запуском выполнено P , то после корректного завершения алгоритма A должно выполняться Q .

Важно не смешивать корректность с эффективностью. Алгоритм может быть корректным, но неприемлемо медленным; быстрый алгоритм может нарушать спецификацию на редком граничном случае.

9. Инвариант цикла

Инвариант цикла — утверждение о состоянии вычисления, которое истинно перед первой итерацией и сохраняется после каждой итерации. Вместе с условием завершения инвариант должен позволять вывести постусловие.

Не любое истинное утверждение полезно. инвариант должен быть достаточно сильным, чтобы вместе с отрицанием условия цикла дать требуемое постусловие, и достаточно простым, чтобы доказать его сохранение.

9.1. Три части доказательства

Шаг	Контрольный вопрос
Инициализация	Почему инвариант истинен до первой итерации?
Сохранение	Почему из истинности инварианта до итерации следует его истинность после выполнения тела?
Завершение	Что следует из инварианта и ложности условия продолжения? Почему это дает постусловие?

9.2. Алгоритм поиска первого достижения порога

Псевдокод

```
FIRST-THRESHOLD( $x, n, \tau$ )  
   $i \leftarrow 0$   
  while  $i < n$  and  $x[i] < \tau$   
     $i \leftarrow i + 1$   
  if  $i = n$   
    return -1  
  return  $i$ 
```

Инвариант $I(i)$: $0 \leq i \leq n$ и для любого j , $0 \leq j < i$, выполняется $x[j] < \tau$.

- Инициализация. При $i = 0$ диапазон $j < i$ пуст, поэтому универсальное утверждение истинно; также $0 \leq i \leq n$ при $n \geq 0$.
- Сохранение. Если цикл выполняется, то $i < n$ и $x[i] < \tau$. После $i \leftarrow i + 1$ новый просмотренный префикс по-прежнему состоит только из значений ниже порога.
- Завершение. Если $i = n$, просмотрен весь массив и подходящего элемента нет. Если $i < n$, ложность $x[i] < \tau$ означает $x[i] \geq \tau$, а инвариант гарантирует, что все предыдущие элементы ниже порога.

10. Доказательство завершения

Для доказательства завершения выбирают вариант цикла — неотрицательную целочисленную величину, которая строго уменьшается на каждой итерации. В сквозном примере подходит $V = n - i$.

- перед каждой итерацией $V > 0$, поскольку условие цикла содержит $i < n$;
- после увеличения i на единицу величина V уменьшается ровно на единицу;
- бесконечная строго убывающая последовательность неотрицательных целых чисел невозможна.

Следовательно, цикл завершится не более чем после n итераций. Здесь впервые появляется количественная информация о числе шагов; формальный анализ трудоемкости будет продолжен в лекции № 2.

11. Рекурсивные алгоритмы и индукция

Для рекурсивного алгоритма роль инварианта часто выполняет индукционное предположение. Нужно определить параметр размера, доказать корректность базового случая, предположить корректность рекурсивных вызовов для меньших размеров и вывести корректность текущего вызова.

Пример рекурсивной суммы

```
SUM(a, n)
  if n = 0
    return 0
  return SUM(a, n - 1) + a[n - 1]
```

Спецификация: при $n \geq 0$ вернуть сумму элементов $a[0..n-1]$. База $n = 0$ дает сумму пустого набора 0. Для $n > 0$ рекурсивный вызов по предположению возвращает сумму первых $n-1$ элементов, после чего прибавляется последний элемент. Завершение следует из строгого уменьшения n .

12. Типичные ошибки спецификации и доказательства

Ошибка	Почему опасна	Как исправить
Постусловие повторяет текст программы	Не фиксирует требуемый результат независимо от реализации.	Описать математические свойства результата.
Не определен пустой вход	Реализация может обратиться к несуществующему элементу.	Явно разрешить или запретить $n = 0$ и задать результат.
Инвариант слишком слаб	Из него и завершения не выводится постусловие.	Добавить свойство обработанного префикса или частичного результата.

Ошибка	Почему опасна	Как исправить
Проверено только несколько тестов	Конкретные примеры не покрывают весь класс входов.	Использовать тесты совместно с доказательством и генерацией граничных случаев.
Нет варианта цикла	Частичная корректность ошибочно принимается за полную.	Указать неотрицательную строго убывающую величину.
Модель вычислений неявна	Сравнение эффективности может оказаться некорректным.	Зафиксировать стоимость операций и особенности данных.

13. Практическое закрепление на лекции

Задание 1. Спецификация

Сформулируйте предусловие и постусловие функции, возвращающей индекс минимального конечного значения массива измерений. Отдельно определите политику для пустого массива, *NaN* и повторяющегося минимума.

Задание 2. Инвариант

Для цикла, вычисляющего сумму первых i элементов массива, предложите инвариант и покажите инициализацию, сохранение и получение постусловия.

Фрагмент для анализа

```
s ← 0
i ← 0
while i < n
    s ← s + a[i]
    i ← i + 1
```

Задание 3. Контрпример

Утверждение: «после завершения последовательного поиска $x[i] = \text{key}$ ». Постройте минимальный контрпример и исправьте постусловие так, чтобы оно учитывало отсутствие ключа.

14. Контрольные вопросы

1. Чем алгоритм отличается от программы и от экземпляра задачи?
2. Какие свойства входят в понятие алгоритма?
3. Зачем отделять спецификацию от реализации?
4. Что описывает предусловие?
5. Что должно быть отражено в постусловии поиска первого подходящего элемента?
6. Чем частичная корректность отличается от полной?
7. Как читается тройка Хоара $\{P\} A \{Q\}$?
8. Что называется инвариантом цикла?
9. Из каких трех частей состоит доказательство с помощью инварианта?
10. Почему инвариант не обязан быть истинным внутри каждого оператора тела цикла?
11. Каким требованиям удовлетворяет вариант цикла?
12. Как доказательство рекурсивного алгоритма связано с математической индукцией?
13. Какие допущения используются в RAM-модели?

14. В каких задачах единичная стоимость доступа к памяти становится слишком грубым допущением?
15. Почему тестирование не заменяет доказательство корректности?

15. Домашнее задание

1. Для алгоритма поиска максимального элемента сформулировать предусловие, постусловие, инвариант и вариант цикла.
2. Реализовать сквозной алгоритм FIRST-THRESHOLD на основном языке практикума. Подготовить тесты: пустой массив, превышение в первом и последнем элементах, отсутствие превышения, равенство порогу, повторные превышения.
3. Для массива измерений с возможными NaN предложить две разные политики обработки и показать, как каждая изменяет спецификацию.
4. Подготовить к лекции № 2 подсчет числа сравнений и обращений к массиву в лучшем и худшем случаях для FIRST-THRESHOLD.

16. Итоги лекции

- Алгоритм решает класс экземпляров, а программа реализует алгоритм в конкретной среде.
- Спецификация задает допустимый вход и обязательные свойства результата.
- Корректность анализируется относительно спецификации и выбранной модели вычислений.
- Полная корректность включает частичную корректность и завершение.
- Инвариант связывает начальное состояние, каждую итерацию и постусловие.
- Вариант цикла доказывает невозможность бесконечного выполнения.

17. Основная и дополнительная литература

1. Основная литература

1. Белов Владимир Викторович, Чистякова Валентина Ивановна. Алгоритмы и структуры данных: Учебник. Москва: ООО "КУРС", 2023, доступ ЭБС
2. Вирт, Н., Ткачев, Ф.В. Алгоритмы и структуры данных: Учебник. Саратов: Профобразование, 2024, доступ ЭБС.
3. Morin P. Open Data Structures. Open textbook, актуальная электронная редакция. Разделы о модели вычислений и базовых структурах данных.

2. Дополнительная литература

4. Sedgewick R., Wayne K. Algorithms. 4th ed. Addison-Wesley, 2011.
5. Goodrich M. T., Tamassia R., Goldwasser M. H. Data Structures and Algorithms in Python. Wiley, 2013.
6. Кнут Д. Э. Искусство программирования. Т. 1–3. М.: Вильямс (действующие издания).

3. Электронные ресурсы

7. Документация Python 3: <https://docs.python.org/3/> (дата обращения: 06.09.2026).
8. cppreference: <https://en.cppreference.com/> (дата обращения: 06.09.2026).
9. ACM/IEEE-CS/AAAI Computer Science Curricula 2023: <https://csed.acm.org/> (дата обращения: 06.09.2026).
10. Open Data Structures: <https://opendatastructures.org/> (дата обращения: 06.09.2026).

4. Программное обеспечение

РЕД ОС; Р7-Офис Профессиональный; Python 3.12 или новее; компилятор GCC/G++ с поддержкой C++20; JupyterLab; Git; VSCodium или Visual Studio Code. Версии быстро меняющихся инструментов указаны по состоянию на 06.09.2026; перед семестром преподаватель проверяет совместимость среды.